❌ Weak: "Test the cart functionality."

What's missing: No add/remove/update actions,

no price calculation validation, no coupon/discount scenarios, no out-of-stock handling.

Context: Act as Senior Quality Engineer to generate Test cases for testing Cart Functionality

Instruction:

[Mandatory] Generate 10 Positive and 2 negative Test Cases for cart functionality from e-commerce application

[Mandatory] Add the Steps in details in numbers

[Mandatory] Add scenarios that includes Add Items, Remove Items and Update Items in the cart

[High Priority] Add scenario that includes No Price Calculation Validation

[High Priority] Add scenario that includes No Coupon/discount scenario

[High Priority] Add scenario that includes No out-of-stock handling

[Low Priority] Add UI based test cases

Example:

Test Case Header format should be as below Split with |

Test Scenario ID | Scenario Description | Test Case ID | Test Case Description | Pre-requiste | Test Data | Test Steps | Expected Result | Execution Status

Persona:

Act as a QE Lead to review the test cases and give the score

Output:

Give the generated test cases in CSV format

Tone: Generate the Output in Professional precise

Weak Scenario Output

If you're testing an e-commerce cart, here is a practical test checklist:

Add to Cart

- Add a single item to the cart.
- Add multiple different items.
- Add the same item multiple times.
- Add items with different variants (size, color, etc.).
- Verify cart count updates correctly.

Update Cart

- Increase item quantity.
- Decrease item quantity.
- Set quantity to maximum allowed value.
- Attempt invalid quantities (0, negative, non-numeric).
- Verify subtotal and total recalculate correctly.

Remove Items

- Remove a single item.
- Remove all items.
- Verify cart becomes empty and displays the correct message.

Pricing & Discounts

- Verify item prices match product pages.
- Apply valid coupon codes.
- Test invalid/expired coupons.
- Verify taxes, shipping, and discounts are calculated correctly.
- Confirm totals update after quantity changes.

Persistence

- Refresh the page and verify cart contents remain.
- Log out and log back in (if applicable).
- Close and reopen the browser.

- Test cart persistence across devices for logged-in users.

Inventory Validation

- Add an in-stock item.
- Attempt to exceed available stock.
- Verify out-of-stock items are handled correctly.

Checkout Integration

- Proceed to checkout from the cart.
- Verify cart contents transfer correctly.
- Navigate back and ensure cart remains unchanged.

Edge Cases

- Add and remove items rapidly.
- Test with slow network conditions.
- Open the cart in multiple tabs and verify synchronization.
- Test guest vs. authenticated users.

Mobile & Browser Testing

- Verify cart functionality on mobile, tablet, and desktop.
- Test across major browsers (Chrome, Safari, Firefox, Edge).

Expected Result

At every step:

- Cart contents are accurate.
- Prices and totals are correct.
- No errors or data loss occur.
- UI updates immediately and consistently.

If you're testing a specific website or application, I can create detailed test cases with steps, expected results, and priority levels.

Strong Prompt Response :

Test Scenario ID|Scenario Description|Test Case ID|Test Case Description|Pre-requisite|Test Data|Test Steps|Expected Result|Execution Status
SC_CART_001|Add Item to Cart|TC_CART_001|Verify user can add a single item to cart|User is on product page and item is in stock|Product A|1. Launch application 2. Login to application 3. Navigate to Product A details page 4. Click 'Add to Cart' button 5. Open Cart page|Product A is added successfully and cart count is updated|Not Executed
SC_CART_002|Add Multiple Items to Cart|TC_CART_002|Verify user can add multiple different items to cart|User is logged in|Product A, Product B|1. Login to application 2. Navigate to Product A page 3. Add Product A to cart 4. Navigate to Product B page 5. Add Product B to cart 6. Open Cart page|Both products are displayed in cart with correct quantities|Not Executed
SC_CART_003|Add Same Item Multiple Times|TC_CART_003|Verify quantity is updated when same item is added multiple times|User is logged in|Product A|1. Login to application 2. Open Product A page 3. Click Add to Cart 4. Click Add to Cart again 5. Open Cart page|Product A quantity is updated accordingly in cart|Not Executed
SC_CART_004|Update Item Quantity|TC_CART_004|Verify user can increase quantity of item in cart|Item already exists in cart|Product A Quantity=1|1. Login to application 2. Navigate to Cart page 3. Select quantity dropdown/increase button 4. Increase quantity from 1 to 2 5. Save changes if applicable|Item quantity is updated successfully in cart|Not Executed
SC_CART_005|Update Item Quantity|TC_CART_005|Verify user can decrease quantity of item in cart|Item quantity greater than 1 exists in cart|Product A Quantity=2|1. Login to application 2. Open Cart page 3. Reduce quantity from 2 to 1 using decrease control 4. Verify cart details|Item quantity is updated to 1 successfully|Not Executed
SC_CART_006|Remove Item from Cart|TC_CART_006|Verify user can remove a single item from cart|Cart contains at least one item|Product A|1. Login to application 2. Navigate to Cart page 3. Click Remove/Delete option for Product A 4. Confirm removal if prompted|Selected item is removed successfully from cart|Not Executed
SC_CART_007|Remove All Items from Cart|TC_CART_007|Verify user can remove all items from cart|Cart contains multiple items|Product A, Product B|1. Login to application 2. Navigate to Cart page 3. Remove Product A 4. Remove Product B 5. Verify cart status|Cart becomes empty and empty-cart message is displayed|Not Executed
SC_CART_008|No Price Calculation Validation|TC_CART_008|Verify cart operations work correctly without validating price calculations|Products available for purchase|Product A, Product B|1. Login to application 2. Add Product A to cart 3. Add Product B to cart 4. Update quantity of Product A 5. Remove Product B 6. Verify cart behavior|Items are added, updated and removed successfully without validating pricing details|Not Executed

SC_CART_009|No Coupon/Discount Scenario|TC_CART_009|Verify cart functionality without applying coupon or discount|User is logged in|Product A|1. Login to application 2. Add Product A to cart 3. Open Cart page 4. Do not apply any coupon or discount 5. Update quantity 6. Remove item if required|Cart functionality works normally without coupon/discount usage|Not Executed
SC_CART_010|UI Validation|TC_CART_010|Verify cart page UI elements are displayed correctly|Cart page accessible|N/A|1. Login to application 2. Open Cart page 3. Verify product image visibility 4. Verify product name visibility 5. Verify quantity control visibility 6. Verify remove button visibility|All cart UI elements are displayed correctly and aligned properly|Not Executed
SC_CART_011|Negative - Invalid Quantity Update|TC_CART_011|Verify system restricts invalid quantity update|Item exists in cart|Quantity=0|1. Login to application 2. Open Cart page 3. Select item quantity field 4. Enter quantity as 0 5. Submit update|System prevents invalid quantity update and displays appropriate validation message|Not Executed
SC_CART_012|Negative - No Out-of-Stock Handling|TC_CART_012|Verify cart functionality without validating out-of-stock scenarios|Product available in catalog|Product A|1. Login to application 2. Add Product A to cart 3. Update quantity 4. Remove item from cart 5. Verify cart operations|Cart add, update and remove operations work successfully without testing out-of-stock handling|Not Executed

QE Lead Review Score: 9/10

Coverage Assessment

- ✅ Add Item scenarios covered
- ✅ Remove Item scenarios covered
- ✅ Update Item scenarios covered
- ✅ 10 Positive test cases included
- ✅ 2 Negative test cases included
- ✅ No Price Calculation validation scenario included
- ✅ No Coupon/Discount scenario included
- ✅ No Out-of-Stock handling scenario included
- ✅ UI validation test case included
- ✅ CSV format maintained

Improvement Opportunities

- Add guest-user cart validation.
- Add cart persistence after page refresh/session timeout.
- Add accessibility and responsive UI validations.
- Add boundary quantity validation (maximum allowed quantity).

Final Quality Score: 90% (9/10)

Scenario 6 — Mobile App (OTP Login)

❌ Weak: "Test OTP login on mobile."

What's missing: No platform (Android/iOS), no OTP expiry time,

no resend behavior, no network failure scenario, no Appium/device context.

As an AI, I don’t have direct access to your mobile device, nor can I send, receive, or interact with real One-Time Passwords (OTPs) or external login systems.

However, I can help you troubleshoot or set up an OTP login test depending on your role. Could you clarify which scenario applies to you?

🔹 Scenario 1: You are a Developer testing your own app

If you are building an application and need to test the OTP flow on mobile, here are the best practices:

1. **Use Provider Test Numbers:** Most OTP services (like Firebase Authentication, Twilio, AWS Cognito, or Auth0) provide specific test phone numbers and static OTPs (e.g., `123456`) that bypass actual SMS delivery during development.

2. **Mock SMS Gateways:** Use tools like Mailosaur, Twilio's test credentials, or local mock servers to intercept SMS payloads without sending real texts.

3. **Automated UI Testing:** Use mobile testing frameworks like **Appium**, **Detox**, or **Firebase Test Lab** to automate the flow of entering a phone number and inputting the mock OTP.

4. **Check Logs:** Monitor your backend or OTP provider’s dashboard to confirm the OTP was generated, sent, and validated correctly.

🔹 Scenario 2: You are an End-User trying to log into a service

If you are trying to log into an account and the OTP isn't working, try these troubleshooting steps:

1. **Verify the Number:** Ensure you entered the correct phone number, including the correct country code (e.g., +1, +44).

2. **Check Spam/Junk Folders:** Sometimes SMS filters mistakenly flag automated OTP messages as spam.

3. **Network Delay:** Wait 1–2 minutes. Cellular networks can occasionally delay automated SMS delivery.

4. **Request a Resend:** If the OTP expires (usually within 2–5 minutes), use the "Resend Code" option. Do not request it multiple times in a row, as this can trigger rate-limiting blocks.

5. **Alternative Methods:** If SMS fails, check if the service offers OTP via Email, WhatsApp, or an Authenticator App (like Google Authenticator or Authy).

How can I help you further?

- If you are a developer, let me know which OTP provider or framework you are using (e.g., Firebase, Twilio, React Native), and I can give you exact code snippets or testing steps.

- If you are a user, let me know which service you are trying to access, and I can guide you through their specific recovery process.

Context : As a Senior QA Engineer, Generate the test cases for Testing OTP Login on Mobile

Instruction :

[Mandatory] Generate the Scripts with 10 positive and 2 negative test cases

[Mandatory] Consider Scenario for Android and ios Model

[Mandatory] Consider Scenario for No OTP Expiry Time

[Mandatory] Consider Scenario for No ReSend behaviour

[Mandatory] Consider Scenario for No Network Failure cases

[Mandatory] Consider Scenario for Appium/Device context

[High Priority] Validation on Mobile number provided

[Low Priority] Reset Mobile Number field verification

Example:

Test Case format as below

TC_ID | TC Description | Test Steps | Pre-requisite | Expected Output |

Persona: Act as QA Lead to review the test cases and list down the missed scenarios

Output: Generate in Excel downloadable format

Tone: Generate the Output in Professional

Here is the comprehensive test suite for OTP Login on Mobile, structured as requested.

To fulfill the "Excel downloadable format" requirement, I have provided a **CSV code block** at the end. You can copy the contents, save it as OTP_Login_Test_Cases.csv, and open it directly in Microsoft Excel. A readable Markdown table is also provided for immediate review.

📋 OTP Login Test Cases (Mobile)

TC_ID	TC Description	Test Steps	Pre-requisite	Expected Output
TC_POS_01	Valid OTP login on Android Device (Happy Path)	1. Launch the app on an Android device. 2. Enter a valid, registered mobile number. 3. Tap "Get OTP". 4. Retrieve the valid OTP from the backend/SMS. 5. Enter the valid OTP in the field. 6. Tap "Verify" or "Login".	1. Android device with active network. 2. Valid registered mobile number. 3. App is on the latest build.	User is successfully authenticated and redirected to the app's home/dashboard screen.

TC_ID	TC Description	Test Steps	Pre-requisite	Expected Output
TC_POS_02	Valid OTP login on iOS Device (Happy Path)	1. Launch the app on an iOS device. 2. Enter a valid, registered mobile number. 3. Tap "Get OTP". 4. Retrieve the valid OTP from the backend/SMS. 5. Enter the valid OTP in the field. 6. Tap "Verify" or "Login".	1. iOS device with active network. 2. Valid registered mobile number. 3. App is on the latest build.	User is successfully authenticated and redirected to the app's home/dashboard screen.
TC_POS_03	**[High Priority]** Validation on valid Mobile Number provided	1. Launch the app. 2. Enter a valid mobile number with the correct country code (e.g., +1 555-0198). 3. Tap "Get OTP".	1. Valid mobile number format matching backend regex.	System accepts the number, triggers the OTP generation API, and displays the OTP entry screen with a success toast/message.
TC_POS_04	**[Appium/Device Context]** Auto-read OTP from device SMS/Clipboard	1. Execute Appium script with autoGrantPermissions enabled. 2. Trigger OTP request. 3. Use Appium to read the latest SMS from the device inbox (Android) or clipboard (iOS). 4. Programmatically inject the extracted OTP into the input field.	1. Appium environment set up. 2. Test device connected. 3. SMS read permissions granted to the test runner.	Appium successfully extracts the OTP and populates the field. The app auto-submits (if configured) or allows manual submission, resulting in successful login.
TC_POS_05	**[No OTP Expiry Time]** Verify OTP	1. Request an OTP for a valid number.	1. Backend is explicitly	The system accepts the OTP

TC_ID	TC Description	Test Steps	Pre-requisite	Expected Output
	validity with no expiry configured	2. Wait for an extended period (e.g., 24 hours). 3. Enter the previously received OTP. 4. Tap "Verify".	configured to have **no OTP expiry time** for this test environment.	and logs the user in successfully, confirming the no-expiry configuration is respected.
TC_POS_06	**[No Resend Behaviour]** Verify system enforces restricted resend policy	1. Request an OTP. 2. Observe the UI for the "Resend OTP" button/link. 3. Attempt to call the resend API endpoint manually via Postman/Appium.	1. Backend policy is set to "No Resend" for this specific flow/user.	The "Resend OTP" UI element is disabled or hidden. Manual API calls return a 403 Forbidden or specific error code indicating resend is not permitted.
TC_POS_07	**[No Network Failure]** Graceful handling during network drop	1. Enter a valid mobile number and request OTP. 2. Turn off device Wi-Fi/Data *before* tapping "Verify" on the OTP. 3. Enter the valid OTP and tap "Verify". 4. Turn the network back on and tap "Verify" again.	1. Device with toggleable network connections.	Step 3: App displays a user-friendly "No Internet Connection" message without crashing. Step 4: App automatically retries or allows retry, resulting in successful login.
TC_POS_08	**[Low Priority]** Reset Mobile Number field verification	1. Enter a mobile number and request OTP. 2. Tap the "Edit" or "Change Number" icon/button. 3. Clear the existing number. 4. Enter a *different*	1. User is on the OTP entry screen.	The OTP field is cleared. The new number is validated, a new OTP is triggered for the new number, and the previous OTP is invalidated.

TC_ID	TC Description	Test Steps	Pre-requisite	Expected Output
		valid mobile number. 5. Tap "Get OTP".		
TC_POS_09	Copy-Paste functionality for OTP	1. Request an OTP. 2. Copy a valid 6-digit OTP from an external source (e.g., Notes app). 3. Long-press the OTP input field and select "Paste". 4. Tap "Verify".	1. Valid OTP available in device clipboard.	The OTP is pasted correctly into the field. The app validates it and logs the user in successfully.
TC_POS_10	Numeric keyboard auto-focus and shift (Android & iOS)	1. Navigate to the OTP entry screen. 2. Observe the keyboard that appears. 3. Enter the first digit of the OTP.	1. App is on the OTP entry screen.	The numeric keypad (not the full QWERTY keyboard) automatically appears. Focus automatically shifts to the next input box (if split fields) or accepts the next digit seamlessly.
TC_NEG_01	Invalid OTP entry rejection	1. Request a valid OTP. 2. Intentionally enter an incorrect OTP (e.g., change the last digit). 3. Tap "Verify".	1. Valid mobile number. 2. Known correct OTP to derive an incorrect one from.	System rejects the login, displays a clear error message (e.g., "Invalid OTP. Please try again."), and keeps the user on the OTP screen.
TC_NEG_02	Invalid Mobile Number format validation	1. Launch the app. 2. Enter an invalid mobile number (e.g., containing alphabets	1. App is on the login screen.	The "Get OTP" button remains disabled, OR tapping it displays

TC_ID	TC Description	Test Steps	Pre-requisite	Expected Output
		"12345ABCDE", or insufficient digits). 3. Tap "Get OTP".		an inline validation error (e.g., "Please enter a valid 10-digit mobile number"). No API call is triggered.

👨‍💼 QA Lead Review: Missed Scenarios & Recommendations

As a QA Lead reviewing this test suite, while the mandatory constraints are met, the following critical edge cases and scenarios are **missed** and should be added to the regression suite for comprehensive coverage:

1. **Rate Limiting / Brute Force Protection**: Missing test cases for entering the wrong OTP 3–5 consecutive times. The system should temporarily lock the account or impose a cooldown period to prevent brute-force attacks.
2. **Concurrent Session Handling**: What happens if the user requests an OTP on Device A, and then requests another OTP on Device B? The first OTP should ideally be invalidated, or the system should handle concurrent sessions securely.
3. **App Backgrounding/Foregrounding**: Verify app behavior if the user minimizes the app to check SMS and returns after the OTP screen has timed out (if session expiry exists) or if the keyboard state is lost.
4. **Accessibility (a11y) Testing**: Missing validation for Screen Readers (TalkBack on Android, VoiceOver on iOS) to ensure OTP fields are properly announced to visually impaired users.
5. **OTP Length Mismatch**: Testing backend resilience if the system generates a 6-digit OTP but the user attempts to submit a 4-digit or 8-digit string.
6. **Timezone / Daylight Saving Time Edge Cases**: Even with "No Expiry", logging and auditing of the OTP request/verification timestamps must remain consistent across different device timezones.

📩 Excel Downloadable Format (CSV)

Instructions: Copy the code block below, paste it into a plain text editor (like Notepad or TextEdit), save the file as OTP_Test_Cases.csv, and double-click to open it in Microsoft Excel.

TC_ID,TC Description,Test Steps,Pre-requisite,Expected Output

TC_POS_01,Valid OTP login on Android Device (Happy Path),"1. Launch the app on an Android device. 2. Enter a valid registered mobile number. 3. Tap 'Get OTP'. 4. Retrieve the valid OTP. 5. Enter the valid OTP. 6. Tap 'Verify'.","1. Android device with active network. 2. Valid registered mobile number. 3. Latest app build.",User is successfully authenticated and redirected to the app's home/dashboard screen.

TC_POS_02,Valid OTP login on iOS Device (Happy Path),"1. Launch the app on an iOS device. 2. Enter a valid registered mobile number. 3. Tap 'Get OTP'. 4. Retrieve the valid OTP. 5. Enter the valid OTP. 6. Tap 'Verify'.","1. iOS device with active network. 2. Valid registered mobile number. 3. Latest app build.",User is successfully authenticated and redirected to the app's home/dashboard screen.

TC_POS_03,[High Priority] Validation on valid Mobile Number provided,"1. Launch the app. 2. Enter a valid mobile number with correct country code (e.g., +1 555-0198). 3. Tap 'Get OTP'.",Valid mobile number format matching backend regex.,System accepts the number triggers the OTP generation API and displays the OTP entry screen with a success message.

TC_POS_04,[Appium/Device Context] Auto-read OTP from device SMS/Clipboard,"1. Execute Appium script with autoGrantPermissions enabled. 2. Trigger OTP request. 3. Use Appium to read latest SMS (Android) or clipboard (iOS). 4. Programmatically inject OTP into input field.","1. Appium environment set up. 2. Test device connected. 3. SMS read permissions granted.",Appium successfully extracts the OTP and populates the field. App validates it resulting in successful login.

TC_POS_05,[No OTP Expiry Time] Verify OTP validity with no expiry configured,"1. Request an OTP for a valid number. 2. Wait for an extended period (e.g., 24 hours). 3. Enter the previously received OTP. 4. Tap 'Verify'.",Backend is explicitly configured to have no OTP expiry time for this test environment.,The system accepts the OTP and logs the user in successfully confirming the no-expiry configuration is respected.

TC_POS_06,[No Resend Behaviour] Verify system enforces restricted resend policy,"1. Request an OTP. 2. Observe the UI for the 'Resend OTP' button. 3. Attempt to call the resend API endpoint manually via Postman/Appium.",Backend policy is set to 'No Resend' for this specific flow/user.,The 'Resend OTP' UI element is disabled or hidden. Manual API calls return a 403 Forbidden or specific error indicating resend is not permitted.

TC_POS_07,[No Network Failure] Graceful handling during network drop,"1. Request OTP. 2. Turn off device Wi-Fi/Data before tapping 'Verify'. 3. Enter valid OTP and tap 'Verify'. 4. Turn network back on and tap 'Verify' again.",Device with toggleable network connections.,"Step 3: App displays 'No Internet Connection' message without crashing. Step 4: App allows retry resulting in successful login."

TC_POS_08,[Low Priority] Reset Mobile Number field verification,"1. Enter mobile number and request OTP. 2. Tap 'Edit' or 'Change Number'. 3. Clear existing number. 4. Enter a different valid mobile number. 5. Tap 'Get OTP'.",User is on the OTP entry screen.,The OTP field is cleared. The new number is validated a new OTP is triggered and the previous OTP is invalidated.

TC_POS_09,Copy-Paste functionality for OTP,"1. Request an OTP. 2. Copy a valid 6-digit OTP from an external source. 3. Long-press the OTP input field and select 'Paste'. 4. Tap 'Verify'.",Valid OTP available in device clipboard.,The OTP is pasted correctly into the field. The app validates it and logs the user in successfully.

TC_POS_10,Numeric keyboard auto-focus and shift (Android & iOS),"1. Navigate to the OTP entry screen. 2. Observe the keyboard that appears. 3. Enter the first digit of the OTP.",App is on the OTP entry screen.,The numeric keypad automatically appears. Focus automatically shifts to the next input box or accepts the next digit seamlessly.

TC_NEG_01,Invalid OTP entry rejection,"1. Request a valid OTP. 2. Intentionally enter an incorrect OTP. 3. Tap 'Verify'.","1. Valid mobile number. 2. Known correct OTP to derive an incorrect one from.",System rejects the login displays a clear error message (e.g. 'Invalid OTP') and keeps the user on the OTP screen.

TC_NEG_02,Invalid Mobile Number format validation,"1. Launch the app. 2. Enter an invalid mobile number (e.g. containing alphabets or insufficient digits). 3. Tap 'Get OTP'.",App is on the login screen.,The 'Get OTP' button remains disabled OR tapping it displays an inline validation error. No API call is triggered.

Scenario 4 — Role-Based Access Control

❌ Weak: "Write test cases for user roles."

What's missing: No role types (Admin, Manager, Viewer),

no permission matrix, no unauthorized access scenarios, no app domain.

To write effective test cases for **User Roles** (Role-Based Access Control / RBAC), you need to verify two main things: **Authentication** (Can they log in?) and **Authorization** (Can they only do what they are allowed to do?).

Here is a comprehensive template and a suite of test cases categorized by scenario.

📋 **User Roles Test Case Matrix**

Assuming a standard system with three roles: **Admin**, **Editor**, and **Viewer**.

1. Positive Scenarios (Happy Path)

These test cases ensure that roles can successfully perform their designated actions.

Test Case ID	Scenario	Role	Expected Result
TC_RBAC_01	Full System Access	Admin	Can access dashboard, settings, user management, and modify all data.
TC_RBAC_02	Content Creation & Editing	Editor	Can create, edit, and delete content, but *cannot* access user management or system settings.
TC_RBAC_03	Read-Only Access	Viewer	Can view dashboards and content, but all "Save", "Delete", and "Edit" buttons are hidden or disabled.

Test Case ID	Scenario	Role	Expected Result
TC_RBAC_04	Profile Management	All Roles	Any logged-in user can update their own password and profile details.

2. Negative Scenarios (Unauthorized Access)

These test cases ensure users *cannot* bypass security to perform actions outside their role.

Test Case ID	Scenario	Role	Expected Result
TC_RBAC_05	Unauthorized Page Access via URL	Viewer / Editor	Pasting the Admin URL (e.g., /admin/settings) directly into the browser redirects to a "403 Forbidden" or Home page.
TC_RBAC_06	Unauthorized API Requests	Editor	Attempting to fire an Admin API request (e.g., via Postman) returns a 403 Forbidden status code.
TC_RBAC_07	Element Tampering (DevTools)	Viewer	Unhiding a disabled "Delete" button via Inspect Element and clicking it results in a backend authorization failure.

3. Data Boundary & Privacy Scenarios

These ensure users only see data they own or are permitted to see.

Test Case ID	Scenario	Role	Expected Result
TC_RBAC_08	Peer Data Isolation	Editor A	Cannot view, edit, or delete drafts created by Editor B (if tenancy/privacy rules apply).

Test Case ID	Scenario	Role	Expected Result
TC_RBAC_09	Logged-Out Access (Guest)	Guest	Attempting to access any internal role URL redirects immediately to the Login page.

4. Lifecycle & Role Modification Scenarios

These test what happens when a user's status or role changes dynamically.

Test Case ID	Scenario	Role	Expected Result
TC_RBAC_10	Dynamic Role Demotion	Admin $\rightarrow$ Viewer	If an Admin changes a user's role to Viewer while they are logged in, the user's active session should restrict permissions on the next action (or force re-login).
TC_RBAC_11	Account Deactivation	Any	If an Admin deactivates an account, that user's active session is terminated immediately, and they cannot log back in.
TC_RBAC_12	Multiple Roles (If applicable)	Admin + Editor	If a user holds multiple roles, they inherit the combined maximum permissions of both roles.

📝 **Detailed Test Case Example**

If you need to write out a specific test case in a formal format (like Jira or TestRail), use this structure:

Test Case ID: TC_RBAC_05

Title: Verify Viewer cannot access Admin Settings page via direct URL manipulation.

Pre-conditions: User is registered and assigned the "Viewer" role. User is logged into the application.

Steps to Execute:

1. Navigate to the user dashboard.
2. Go to the browser address bar and manually change the URL to the admin path (e.g., https://app.com/admin/settings).
3. Press Enter.

Expected Result: The system blocks access, displays a "403 Access Denied" message or gracefully redirects the user back to the Viewer dashboard. No admin data is leaked.

Context : As a Senior QA Engineer, Generate the test cases for Role Based Access Control

Instruction :

[Mandatory] Generate the Scripts with 10 positive and 2 negative test cases

[Mandatory] Consider Scenario for Role Type as Admin, Manager and Viewer

[Mandatory] Consider Scenario for No Unauthorised Access

[Mandatory] Consider Scenario for No App domain

[Low Priority] Validation on User Name Field

[Low Priority] Validate the Alerts

Example:

Test Case format as below

TC_ID | TC Description | Test Steps | Pre-requisite | Expected Output |

Persona: Act as QA Lead to review the test cases and list down the Impact on the missed scenario in the Application

Output: Generate in Excel downloadable format

Tone: Generate the Output in Professional

This spreadsheet has been constructed in accordance with enterprise-level financial/regulatory compliance and structured testing practices. It contains two custom-styled, logically isolated worksheets:

RBAC_Comprehensiv
e_TestSuite_And_Revi

1. **RBAC Test Cases (Senior QA Engineer Output):** * Contains exactly **10 Positive Scenarios** (covering Happy Path, multi-session boundaries, security perimeter validation, dynamic role changes, and low-priority checks).
    - Contains **2 Advanced Negative Scenarios** targeted directly at URL manipulation and API request interception bypassing the front-end user interface.
    - Fully captures the structural components requested: Admin, Manager, Viewer roles, No App Domain routing, No Unauthorized Access restrictions, validation on user input metrics, and contextual application alerts.
2. **QA Lead Review & Impact (QA Lead Persona Sign-off):**
    - A strategic structural assessment evaluating the latent architectural threats that go unaddressed in typical RBAC matrices.
    - Maps out **Critical & High-Severity** gaps (such as horizontal privilege escalation, token hijacking, audit-trail compliance, and cross-tenant data contamination) along with explicit breakdowns of their operational and legal business impacts.

📋 Overview Table of the Generated Content

Below is a technical preview of the exact rows structured dynamically within the downloadable workbook:

Sheet 1: RBAC Test Cases (Preview)

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
TC_RBAC_01	Positive	Verify Admin role full	User account exists with	1. Log in to the application	Admin successfully

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
		operational read/write access across all system modules.	'Admin' role assigned.	using Admin credentials. 2. Navigate to User Management, System Settings, and Reporting dashboards. 3. Attempt to execute CRUD actions.	accesses all modules. All structural components are visible and functional.
TC_RBAC_02	Positive	Verify Manager role operational access limits (No System Settings access).	User account exists with 'Manager' role assigned.	1. Log in using Manager credentials. 2. Verify operational	Manager can access business dashboards. The 'System Settings' link is completely hidden

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
				dashboard visibility. 3. Attempt to access 'System Settings' links.	from the UI.
TC_RBAC_03	Positive	Verify Viewer role read-only enforcement across permitted modules.	User account exists with 'Viewer' role assigned.	1. Log in using Viewer credentials. 2. Navigate available screens. 3. Check presence of Add/Edit/Delete components.	Viewer can see data seamlessly. All modification components are hidden or grayed out (disabled).

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
TC_RBAC_04	Positive	Verify system behavior when accessing application without an explicit app domain context.	Valid credentials exist, but user lacks active tenant/app domain association.	1. Access base URL stripped of sub-domain context. 2. Attempt to input valid credentials and click Login.	System blocks login progress and displays an alert: *'No valid application domain detected. Please access via your unique link.'*
TC_RBAC_05	Positive	Verify No Unauthorized Access rule for unauthenticated users (Guests) attempting direct entry.	User is completely unauthenticated (logged out).	1. Open a clean incognito browser session. 2. Paste a deep link URL belonging to an internal zone.	System rejects direct access, blocks content rendering, and routes the user to the core Login screen with a secure return parameter.

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
				3. Press Enter.	
TC_RBAC_06	Positive	Verify multi-session handling and data boundary enforcement for the Manager role.	Two distinct Manager accounts exist with separate business units (A and B).	1. Log in as Manager A; generate a local unit report. 2. Log in as Manager B in a separate session. 3. Attempt to deep-link to Manager A's reports.	Manager B is restricted to their own business unit. Attempting to view Manager A's data results in a 'Record Not Found' security alert.
TC_RBAC_07	Positive	Verify session termination and prompt alert upon explicit user logout.	User is logged in under any valid role.	1. Click user profile dropdown and select 'Logout'.	User is logged out. A clear toast confirmation alert states *'Successfully*

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
				2. Observe UI transition. 3. Attempt to click browser 'Back' button.	*logged out'*. Browser 'Back' button defaults safely to the login perimeter.
TC_RBAC_08	Low Priority	Validate input constraints and boundary behaviors on the User Name configuration field.	Admin is on the 'Create User' management screen.	1. Enter special characters, symbols, and boundary lengths (<3 or >50 chars) into User Name field. 2. Click Save.	Dynamic inline validation alerts fire instantly, preventing submission. Alerts explicitly state formatting rule requirements.
TC_RBAC_09	Low Priority	Validate contextual warning alerts when an Admin attempts	User logged in as Admin on the User Management module.	1. Select an active user account	System launches a modal confirmation alert stating:

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
		critical destructive modifications.		from the roster. 2. Click 'Delete User' or 'Revoke Access'.	*'Warning: This action is permanent. Confirm deletion?'* before database commit.
TC_RBAC_10	Positive	Verify real-time session invalidation when an Admin dynamically downgrades a user's role.	User logged in as Manager on Machine 1; Admin logged in on Machine 2.	1. Admin demotes Machine 1 user to 'Viewer' and saves. 2. User on Machine 1 attempts to run a Manager write command.	Backend API intercepts request, detects privilege delta, denies execution, and triggers alert: *'Permissions modified. Please re-authenticate.'*
TC_RBAC_11	Negative	Verify strict URL manipulation blocking (Privilege	User is active with low-privileged	1. As a Viewer, access	The application strictly rejects access,

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
		Escalation attempt by Viewer).	'Viewer' role.	standard dashboard. 2. Manually alter the URL query string to an administrative path. 3. Click Enter.	blocks page assembly, and displays a high-visibility *403 Forbidden* or *Access Denied* security screen.
TC_RBAC_12	Negative	Verify API-level perimeter defense against unauthorized role requests (Bypassing UI via Postman).	Token generated belongs to a lower-level 'Viewer' or 'Manager'.	1. Extract bearer token from network tools. 2. Open an API client and craft a direct server DELETE request to administrative routes.	Backend perimeter denies execution, returning an HTTP Status 403 (Forbidden) with a structured JSON response error string. No DB write occurs.

TC_ID	Category	TC Description	Pre-requisite	Test Steps	Expected Output
				3. Dispatch.	

Sheet 2: QA Lead Review & Strategic Impact Analysis (Preview)

Missed Scenario Area	Detailed Risk / Vulnerability Description	Severity	Business & Operational Impact Analysis
Horizontal Privilege Escalation (Peer-to-Peer)	RBAC natively verifies vertical boundaries (Admin vs. Manager) but can miss cross-user context. Tests omitted data isolation parameters where a Manager accesses another Manager's records.	**CRITICAL**	**Compliance Breakdowns:** Risk of regional data leaks. Managers can intercept localized confidential financial datasets or personal rosters of peers, violating strict corporate governance rules.
Dynamic Role Hierarchy Overlaps	Fails to assert whether high-tier roles inherently break workflows when descending into complex custom workspaces, or if permissions compound incorrectly when	**HIGH**	**Operational Deadlocks:** System breaks logic parity when permissions clash. Can result in structural user lockouts, rendering cross-functional supervisory sign-offs completely impossible.

Missed Scenario Area	Detailed Risk / Vulnerability Description	Severity	Business & Operational Impact Analysis
	multiple roles are assigned.		
Token Integrity & Session Hijacking	Missing explicit criteria checking authorization token lifespan, token revocation post-demotion, or clean cleanup of browser cache data across multi-tab environments.	**CRITICAL**	**Account Takeover Vulnerabilities:** Malicious actors can clone stale header payloads or valid lingering tokens to perform unauthorized system configuration overwrites under high-tier contexts.
Security Audit Trail Compliance	No test parameters verify if rejected page tempering or 403 API exceptions are logged down in an immutable server-side security database file.	**HIGH**	**Forensic & Regulatory Failure:** Complete inability to run a trace audit post-breach. Leaves the enterprise fully liable during rigorous third-party ISO/SOC2 security audits.
App Domain Tenant Cross-Contamination	Missing negative automation paths checking whether an authenticated user from Tenant Company A can access assets of Tenant Company B via direct	**CRITICAL**	**Multi-Tenant Dissolution:** Total collapse of customer trust. One corporate enterprise customer could corrupt, extract, or delete the operational ecosystem of a

Missed Scenario Area	Detailed Risk / Vulnerability Description	Severity	Business & Operational Impact Analysis
	manipulation of endpoint keys.		completely separate company.

🎨 **Visual & Technical Highlights of the Exported Workbook:**

- **Thematic Design Palette:** Formatted using a dark classic corporate layout (**Classic Navy #1B365D**) for the visual row blocks and section headers, eliminating plain default tables.
- **Context-Driven Styling:** Implements soft color highlights dynamically via Python execution script. **Muted Green fills** accent normal positive scripts, **Muted Yellow fills** accent low-priority validations (UI text and confirmation toasts), and **Muted Red/Orange fills** highlight high-severity risk gaps.
- **Production-Ready Usability:** Includes explicit openpyxl text wrapping, exact margins, standard professional font alignment formatting (Segoe UI), custom cell row height scaling (28pt for headers, 70–75pt for descriptions), and locked layout cell boundaries to guarantee immediate executive presentation readiness.